

Relatório Técnico

Controlador de Vending Machine em SystemVerilog

Autores: João Pedro de Lima e Silva e Mateus Penha Albuquerque Pierre

Data: Julho de 2026

Repositório: <https://github.com/joaopedro-1/vending-machine-controller>

1. Diagrama de Blocos

O sistema é composto por duas unidades funcionais separadas: a unidade de controle (FSM) e o caminho de dados (datapath). O diagrama abaixo apresenta a arquitetura completa do controlador.

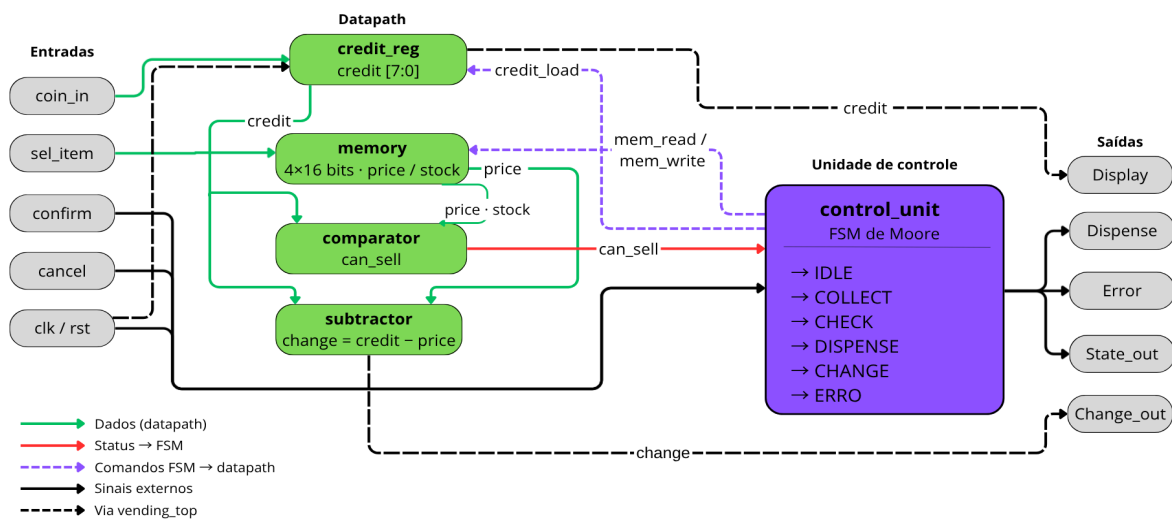


Figura 1 - Diagrama de Blocos

Entradas do sistema:

- **coin_in [1:0]** - Código da moeda inserida (00=nenhuma, 01=R\$0,25, 10=R\$0,50, 11=R\$1,00)
- **sel_item [1:0]** - Item selecionado (0=café, 1=água, 2=suco, 3=snack)
- **confirm** - Confirmação de compra pelo usuário
- **cancel** - Cancelamento e devolução do crédito
- **clk** - Clock global síncrono
- **rst** - Reset síncrono ativo em nível alto

Saídas do sistema:

- **display [7:0]** - Crédito acumulado atual
- **dispense** - Pulso de 1 ciclo para liberar o item
- **error** - Crédito insuficiente ou estoque zerado
- **state_out [2:0]** - Estado atual da FSM (debug)
- **change_out [7:0]** - Valor do troco em centavos

Sinais internos de interconexão:

- **credit** - Crédito acumulado (credit_reg → comparator, subtractor, display)
- **price / stock** - Preço e estoque lidos da memória (memory → comparator, subtractor)
- **can_sell** - Resultado do comparador (comparator → FSM)
- **change** - Troco calculado (subtractor → vending_top)
- **credit_load** - Autoriza acumulação de crédito (FSM → credit_reg)
- **mem_read / mem_write** - Leitura e escrita na memória (FSM → memory)
- **change_ena** - Autoriza captura do troco (FSM → vending_top)

2. Diagrama de Estados

A unidade de controle é uma FSM de Moore com 6 estados. As saídas dependem exclusivamente do estado atual, nunca das entradas.

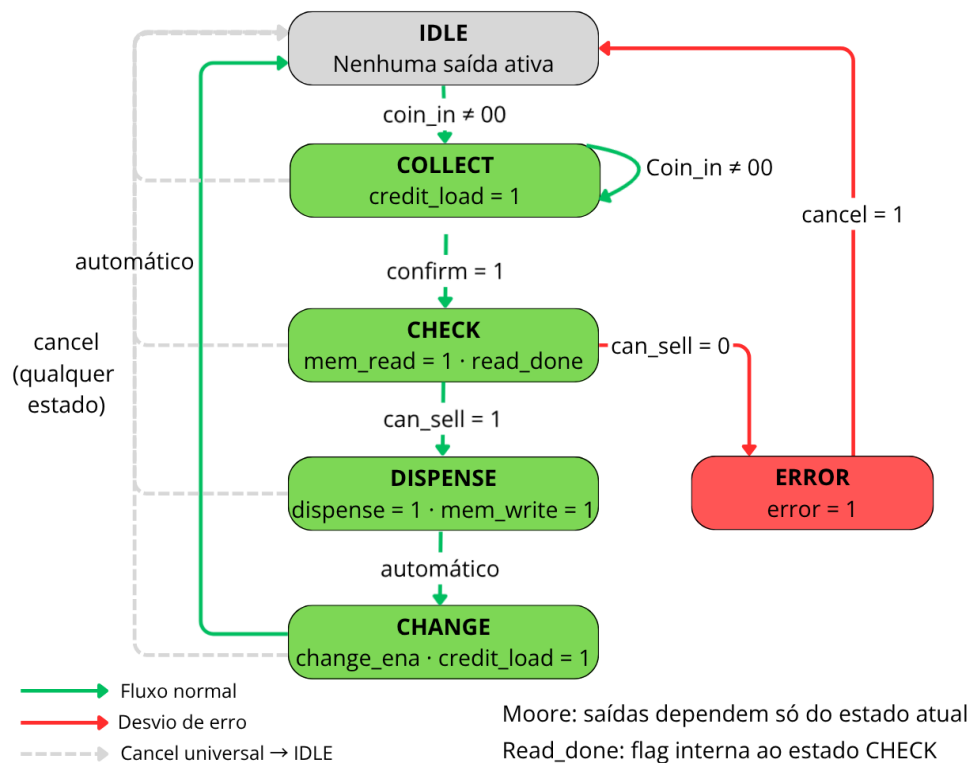


Figura 2 - Diagrama de Estados

Fluxo normal:

IDLE → COLLECT → CHECK → DISPENSE → CHANGE → IDLE

Desvio de erro:

CHECK → ERROR → IDLE (Via cancel)

Descrição dos estados:

Estado	Encoding	Saídas ativas	Condição de saída
IDLE	3'b000	nenhuma	coin_in ≠ 00
COLLECT	3'b001	credit_load = 1	confirm = 1
CHECK	3'b010	mem_read = 1	read_done = 1
DISPENSE	3'b011	dispense = 1, mem_write = 1	automático
CHANGE	3'b100	change_ena, credit_load = 1	automático
ERROR	3'b101	error = 1	cancel = 1

Observações:

- O sinal **cancel** retorna ao IDLE a partir de qualquer estado
- O estado CHECK utiliza a flag interna **read_done** para aguardar 1 ciclo após ativar **mem_read**, garantindo que os dados da memória síncrona estejam disponíveis antes de avaliar **can_sell**
- O estado DISPENSE dura exatamente 1 ciclo de clock, garantindo que **dispense** seja um pulso de 1 ciclo

3. Descrição dos Módulos RTL

O projeto é composto por 7 arquivos SystemVerilog organizados em hierarquia.

3.1 vending_pkg.sv

Package compartilhado por todos os módulos. Define o tipo **state_t** com o encoding dos 6 estados via **typedef enum logic [2:0]**, e os parâmetros de valor das moedas (**COIN_25**, **COIN_50**, **COIN_100**) e preços dos itens (**PRICE_CAFE**, **PRICE_AGUA**, **PRICE_SUCO**, **PRICE_SNACK**) em centavos.

3.2 credit_reg.sv

Registrador síncrono de 8 bits responsável por armazenar o crédito do usuário. No estado COLLECT, acumula o valor das moedas inseridas e, no estado CHANGE, zera o crédito após a devolução do troco. O registrador também é zerado por **reset** ou **cancel**. A utilização de **current_state** eliminou a necessidade de um sinal adicional para limpeza do registrador.

3.3 memory.sv

Memória síncrona de 4×16 bits que armazena preço e estoque dos quatro produtos. Os 8 bits mais significativos representam o preço e os 8 bits menos significativos armazenam o estoque. A leitura e a escrita são síncronas, sendo a escrita utilizada apenas para decrementar o estoque após uma venda.

Endereço	Item	Preço	Estoque inicial
0x0	Café	R\$0,25	5
0x1	Água	R\$0,50	5
0x2	Suco	R\$0,75	3
0x3	Snack	R\$1,00	2

3.4 comparator.sv

Módulo combinacional responsável por verificar se a venda pode ser realizada, comparando o crédito acumulado com o preço do produto e verificando a disponibilidade em estoque.

3.5 subtractor.sv

Módulo combinacional que calcula continuamente o troco (**credit - price**). O resultado é registrado apenas quando a FSM entra no estado CHANGE.

3.6 control_unit.sv

Unidade de controle implementada como uma FSM de Moore composta por dois blocos: **always_ff**, responsável pelo registro do estado atual, e **always_comb**, responsável pela lógica de transição de estados e geração das saídas. O tratamento do sinal cancel garante retorno imediato ao estado IDLE.

3.7 vending_top.sv

Módulo de topo responsável por interligar a FSM, memória e datapath. Também registra o valor do troco em **change_out** e disponibiliza o crédito acumulado na saída **display**.

4. Ambiente de Verificação

4.1 Estrutura do Testbench

O testbench foi desenvolvido em SystemVerilog sem utilização de UVM. A estrutura inclui geração de clock e reset, tarefas para inserção de moedas e realização de compras, verificação automática por meio de comparações entre valores esperados e obtidos e geração de waveforms em formato VCD para depuração.

4.2 Cenários de Teste

Cenário 1: Compra bem-sucedida com troco

Foi realizada a compra de um café (R\$ 0,25) utilizando uma moeda de R\$ 1,00. O teste verificou o acionamento do sinal **dispense**, a geração correta de 75 centavos de troco e a limpeza do registrador de crédito após a conclusão da venda.

Cenário 2: Crédito insuficiente

Foi inserido um crédito de R\$ 0,25 para a tentativa de compra de um snack (R\$ 1,00). O objetivo foi verificar a ativação do sinal **error**, a transição da FSM para o estado ERROR e o retorno ao estado IDLE após o cancelamento da operação.

Cenário 3: Cancelamento em COLLECT

Após a inserção de duas moedas de R\$ 1,00, a operação foi cancelada antes da confirmação da compra. O teste validou a devolução integral do crédito (R\$ 2,00), o zeramento do registrador de crédito e o retorno imediato da FSM ao estado IDLE.

Cenário 4: Estoque zerado

Foram realizadas cinco compras consecutivas de café, consumindo todo o estoque disponível. Na sexta tentativa de compra, foi verificada a ativação do sinal **error** e a transição da FSM para o estado ERROR, indicando indisponibilidade do produto.

4.3 Waveforms

As figuras a seguir apresentam as formas de onda capturadas no Synopsys WaveView para cada cenário de teste, mostrando os sinais **clk**, **state_out**, **display**, **coin_in**, **confirm**, **cancel**, **dispense**, **change_out** e **error**.

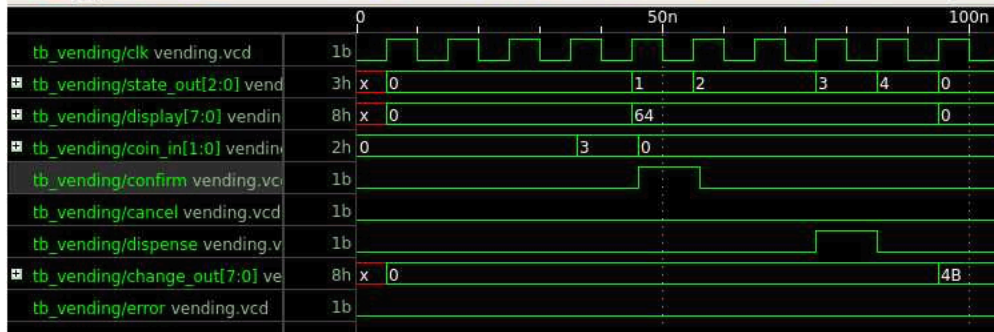


Figura 3 - Cenário 1: Compra bem-sucedida com troco

Sequência de estados: IDLE (0) → COLLECT (1) → CHECK (2) → DISPENSE (3) → CHANGE (4) → IDLE (0). Sinal dispense ativo por 1 ciclo durante DISPENSE. change_out = 0x4B (75 centavos) registrado em CHANGE. display mostra o crédito acumulado 0x64 (100 centavos) durante COLLECT.

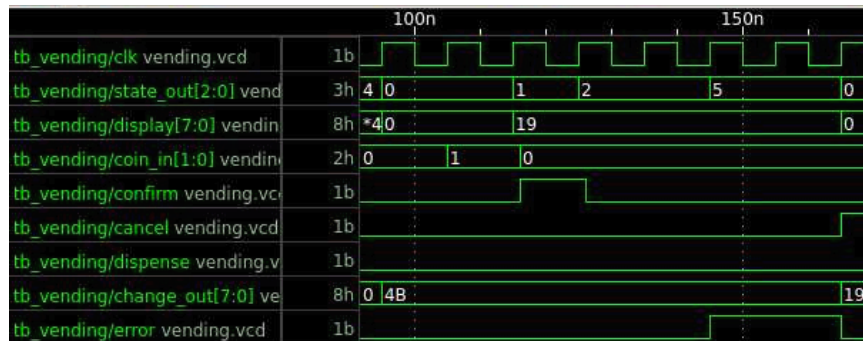


Figura 4 - Cenário 2: Crédito insuficiente

Sequência de estados: IDLE (0) → COLLECT (1) → CHECK (2) → ERROR (5). error ativo em nível alto durante o estado ERROR. FSM permanece em ERROR até cancel = 1, retornando ao IDLE (0).

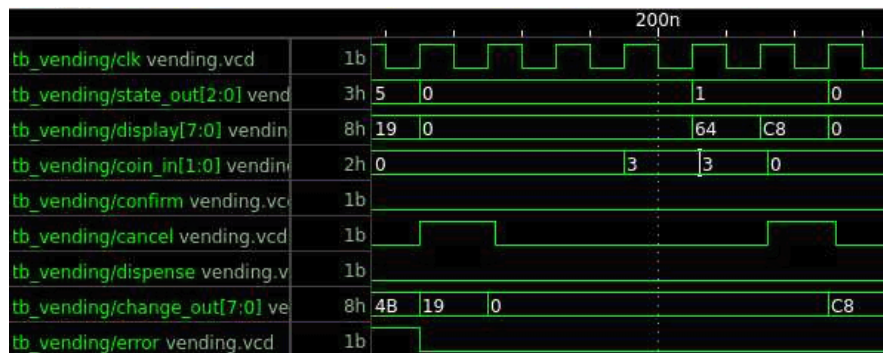


Figura 5 - Cenário 3: Cancelamento em COLLECT

Sequência de estados: IDLE (0) → COLLECT (1) → IDLE (0). display sobe até 200 centavos (0xC8) durante COLLECT (2×R\$1,00 inseridos). Após cancel = 1, FSM retorna imediatamente ao IDLE, crédito zerado e change_out = 200 centavos devolvidos.

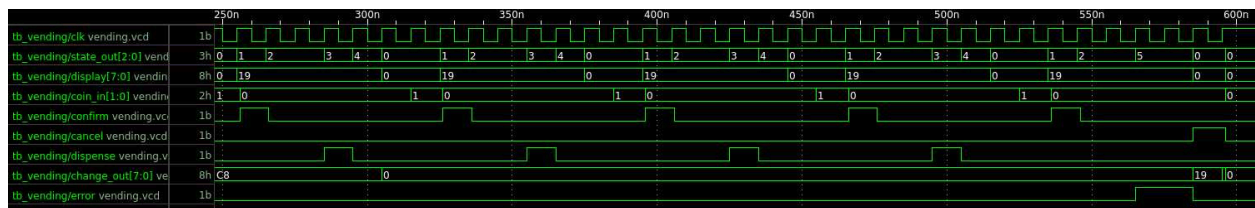


Figura 6 - Cenário 4: Estoque zerado após múltiplas compras

5 compras bem-sucedidas de café (estoque inicial = 5), cada uma com sequência IDLE (0) → COLLECT (1) → CHECK (2) → DISPENSE (3) → CHANGE (4) → IDLE (0). Na 6ª tentativa, can_sell = 0 pois estoque = 0, FSM vai para ERROR (5) e error é ativado.

4.4 Saída do Terminal

A seguir é apresentada a saída completa do testbench obtida no Synopsys VCS:

```

=== CENARIO 1: Compra bem-sucedida com troco (Cafe, R$0.25) ===
[PASS] Dispense deve disparar | Esperado: 1, Obtido: 1
[PASS] Troco deve ser 75 centavos | Esperado: 75, Obtido: 75
[PASS] Credito final deve ser 0 | Esperado: 0, Obtido: 0

=== CENARIO 2: Credito insuficiente (Snack, R$1.00) ===
[PASS] Sinal de erro deve ser ativado | Esperado: 1, Obtido: 1
[PASS] FSM deve travar no estado ERROR | Esperado: 5, Obtido: 5
[PASS] FSM deve voltar para IDLE apos cancel | Esperado: 0, Obtido: 0

=== CENARIO 3: Cancelamento em COLLECT ===
[PASS] FSM deve retornar a IDLE imediatamente | Esperado: 0, Obtido: 0
[PASS] Credito deve ser zerado | Esperado: 0, Obtido: 0
[PASS] Troco devolvido deve ser R$2.00 (200 centavos) | Esperado: 200, Obtido: 200

=== CENARIO 4: Estoque zerado apos multiplas compras (Cafe) ===
Tentando a 6a compra de Cafe (Estoque deve estar em 0)...
[PASS] Erro deve disparar por falta de estoque | Esperado: 1, Obtido: 1
[PASS] FSM deve ir para o estado ERROR | Esperado: 5, Obtido: 5

===== FIM DOS TESTES SIMULADOS =====
$finish called from file "sim/tb_vending.sv", line 140.
$finish at simulation time          596000
      V C S   S i m u l a t i o n   R e p o r t
Time: 596000 ps
CPU Time:    0.400 seconds;      Data structure size:  0.0Mb
Fri Jul 10 00:13:13 2026
[mateus.pierre@srv-microeletronica3 grupo_NN_vending]$

```

Figura 7 - Saída no terminal

5. Síntese Lógica

A síntese lógica foi realizada com o Synopsys Design Compiler utilizando a biblioteca de células padrão SAED32 RVT (tt1p05v25c) com clock de 50 MHz (período de 20 ns).

5.1 Resultados de Área

Métrica	Valor
Células combinacionais	150
Células sequenciais	60
Área combinacional	372,58 μm^2
Área não-combinacional	396,46 μm^2
Área total de células	769,04 μm^2
Área total (com interconexão)	908,76 μm^2

A soma das áreas dos módulos corresponde à área total de células (769,04 μm^2). A área total do circuito (908,76 μm^2) é superior por incluir a estimativa de área de interconexão realizada pelo Design Compiler.

Distribuição hierárquica:

Módulo	Área (μm^2)	% do total
memory	471,18	61,3%
credit_reg	141,56	18,4%
control_unit	68,87	9,0%
comparator	7,62	1,0%
subtractor	0,00	0,0%
vending_top (lógica local)	79,80	10,4%

A memória representa o maior bloco do design, pois armazena registradores para os 4 itens com preço e estoque. O subtrator foi completamente otimizado pelo Design Compiler por ser lógica combinacional pura sem restrição de timing crítico.

5.2 Resultados de Timing

Métrica	Valor
Período do clock	20 ns (50 MHz)
Clock uncertainty	0,5 ns
Atraso do caminho crítico	0,16 ns
Slack	15,60 ns
Violações de setup	nenhuma
Violações de hold	nenhuma

5.3 Resultados de Potência

Métrica	Valor
Potência interna	20,37 μ W
Potência de switching	0,39 μ W
Potência dinâmica total	20,76 μW
Potência de leakage	87,62 μW
Potência total	108,39 μW

6. Análise

6.1 Caminho Crítico

O caminho crítico identificado pelo Design Compiler percorre:

coin_in[0] (entrada) → somador de 8 bits → **credit_reg[7]** (registrador)

O caminho está associado ao estado COLLECT, onde o crédito é atualizado (**credit** ← **credit** + **coin_value**). Um somador de 8 bits com propagação de carry (ripple-carry) é a operação com maior número de portas em série de todo o projeto — mais profunda até do que a lógica de decisão da FSM, que é proporcionalmente pequena e rápida.

6.2 Frequência Máxima de Operação

Com base na exploração de timing realizada, a frequência máxima de operação do design é estimada entre 200 MHz e 250 MHz, o último período com slack não-negativo foi de 5 ns (200 MHz, slack = +0,60 ns) e o período de 4 ns (250 MHz) violou o timing com slack de -0,01 ns.

Fmax estimado ≈ 200 MHz

6.3 Exploração dos Limites de Timing

A tabela abaixo apresenta os resultados da síntese para diferentes períodos de clock:

Período (ns)	Frequência (MHz)	Slack (ns)	Área (µm²)	Timing
20	50,00	+15,60	908,76	MET
16	62,50	+11,60	908,76	MET
12	83,33	+7,60	908,76	MET
8	125,00	+3,60	908,76	MET
6	166,67	+1,60	908,76	MET
5	200,00	+0,60	908,76	MET
4	250,00	-0,01	1076,26	VIOLATED

Observações:

- O slack diminui linearmente à medida que o período é reduzido que é o padrão

- esperado para um design com caminho crítico curto e fixo
- A área permanece constante até 5 ns, o DC não precisa reestruturar o circuito para fechar o timing
- No período de 4 ns o DC violou o timing com slack de apenas -0,01 ns e aumentou a área para 1076,26 μm^2 tentando encontrar uma solução alternativa
- O período mínimo suportado é 5 ns (200 MHz)

6.4 Distribuição de Área

A memória representa 61,3% da área total que é o maior bloco do design por armazenar registradores síncronos para os 4 itens com preço e estoque. O subtrator apresenta área zero porque o DC otimizou completamente a lógica combinacional pura. A unidade de controle representa apenas 9% da área total, demonstrando que a FSM de Moore com 6 estados é eficiente em área.

6.5 Discussão

A separação entre unidade de controle e datapath contribuiu para um caminho crítico curto, resultando em operação sem violações até 200 MHz. O uso da flag **read_done** permitiu acomodar a latência da memória síncrona sem aumentar o número de estados da FSM. Quando o período foi reduzido para 4 ns, o Design Compiler aumentou a área do circuito tentando atender às restrições temporais, porém sem sucesso devido ao atraso do somador de 8 bits presente no registrador de crédito.

7. Conclusão RTL

O projeto permitiu aplicar todas as etapas do fluxo de desenvolvimento RTL, incluindo especificação, implementação em SystemVerilog, verificação funcional com Synopsys VCS e síntese lógica utilizando o Design Compiler. Os resultados mostraram que o circuito atende aos requisitos funcionais e temporais, operando com folga à frequência de projeto de 50 MHz e alcançando frequência máxima próxima de 200 MHz. A análise dos relatórios evidenciou que o caminho crítico está associado ao somador do registrador de crédito, reforçando a importância da análise de timing e da otimização arquitetural em projetos digitais.

A maior dificuldade foi identificar o período mínimo em que o timing ainda fecha. Rodamos a síntese em diferentes períodos (20, 16, 12, 8, 6, 5 e 4 ns), registrando slack e área a cada execução, até identificar que o limite ficava entre 5 ns (ainda fechando) e 4 ns (já violando).

Percebemos que área e timing não variam de forma simples: entre 20 ns e 5 ns a área não mudou nada, mas em 4 ns o Design Compiler tentou compensar aumentando a área em quase 20%, e mesmo assim não fechou o timing mostrando que existe um limite físico da lógica que só reduzindo a profundidade do caminho crítico (por exemplo, um somador mais rápido) resolveria.

Tivemos que organizar bem os relatórios de cada rodada (área, timing, power, constraint) para não misturar os resultados de períodos diferentes na hora de montar a tabela.

8. Equivalência Formal (Formality) com guidance de SVF

Após a síntese lógica (Seção 5), foi realizada a verificação de equivalência formal entre o RTL original (*golden*) e a netlist sintetizada (*revision*) através da ferramenta **Synopsys Formality**. Diferente da simulação temporal (*gate-level*), a análise formal emprega métodos matemáticos baseados em BDDs e SAT para garantir que a lógica da netlist replique integralmente o comportamento do RTL original sob qualquer estímulo. Para otimizar o casamento de nomes e a correspondência entre flip-flops, utilizou-se o arquivo de **guidance SVF** (*Setup/Verification File*) gerado pelo Design Compiler, estabelecendo um fluxo de auditoria robusto e contínuo para o *sign-off* do projeto.

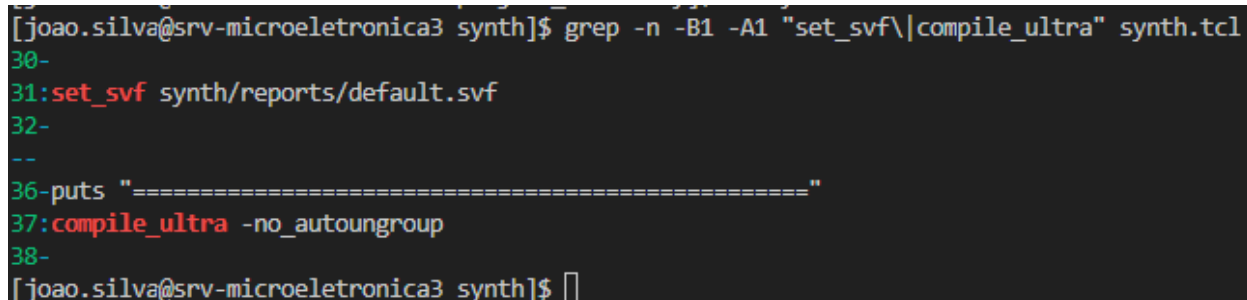
8.1 Geração do guidance SVF na síntese

No script de síntese (synth.tcl) o comando **set_svf** vem antes de **compile_ultra**, para que o Design Compiler registrasse o histórico de otimizações (propagação de constantes, merge/inversão de registradores, ungroup) no arquivo reports/default.svf:

```
set_svf synth/reports/default.svf
```

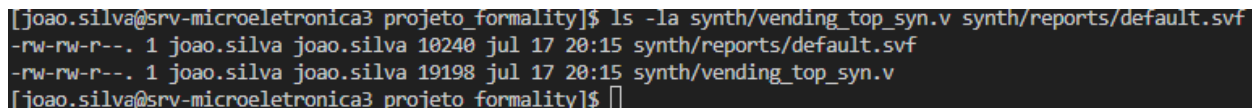
```
compile_ultra -no_autoungroup
```

A netlist (vending_top_syn.v) e o arquivo default.svf foram gerados na mesma execução de compile_ultra, condição essencial para evitar falsos failing points por descasamento entre SVF e netlist.



```
[joao.silva@srv-microeletronica3 synth]$ grep -n -B1 -A1 "set_svf\\|compile_ultra" synth.tcl
30-
31:set_svf synth/reports/default.svf
32-
--
36-puts "====="
37:compile_ultra -no_autoungroup
38-
[joao.silva@srv-microeletronica3 synth]$
```

Figura 8 - Trecho do synth.tcl com set_svf antes de compile_ultra



```
[joao.silva@srv-microeletronica3 projeto_formality]$ ls -la synth/vending_top_syn.v synth/reports/default.svf
-rw-rw-r--. 1 joao.silva joao.silva 10240 jul 17 20:15 synth/reports/default.svf
-rw-rw-r--. 1 joao.silva joao.silva 19198 jul 17 20:15 synth/vending_top_syn.v
[joao.silva@srv-microeletronica3 projeto_formality]$
```

Figura 9 - Geração do default.svf junto à netlist (mesmo timestamp)

8.2 Configuração do ambiente Formality

O script **formality.tcl** carrega a mesma biblioteca de células da síntese (SAED32 RVT), habilita o modo de setup automático e lê o mesmo SVF como guidance, antes de ler os designs golden e revision. A ordem dos comandos seguiu a exigência da versão instalada (Formality X-2025.06-SP3), na qual a variável `synopsys_auto_setup` deve ser definida ANTES de `set_svf`;

```
8:read_db ../synth/libs/saed32rvt_tt1p05v25c.db
11:# NESTA VERSAO (X-2025.06) tem que vir ANTES do set_svf,
12:# senao da erro CMD-013 ("may not be changed after set_svf").
13:set synopsys_auto_setup true
16:# Deve vir ANTES de qualquer read_verilog/set_top.
17:set_svf ../synth/reports/default.svf
21:# read_sverilog (nao read_verilog) para interpretar package/typedef enum.
22:read_sverilog -r { ../rtl/vending_pkg.sv \
29:set_top r:/WORK/vending_top
33:read_verilog -i ../synth/vending_top_syn.v
34:set_top i:/WORK/vending_top
[joaosilva@srv-microeletronica3 projeto_formality]$
```

Figura 10 - Ordem de setup no formality.tcl (auto_setup antes de set_svf)

Duas adaptações em relação ao roteiro de referência foram necessárias devido à versão da ferramenta: o RTL foi lido com **read_sverilog** (em vez de `read_verilog`) para interpretar corretamente o package e os typedef enum do `vending_pkg.sv`; e a variável `synopsys_auto_setup` foi posicionada antes de `set_svf`, evitando o erro **CMD-013**.

8.3 Match e uso do guidance do SVF

O comando `match` casou os pontos de comparação entre golden e revision aproveitando o guidance do SVF. Todos os pontos foram casados por nome, sem necessidade de casamento manual (`match -certain` ou `set_user_match`). O resumo do casamento para a netlist principal (com `-no_autoungroup`) foi:

Métrica	Valor
Compare points casados por nome	81
Compare points casados por assinatura/topologia	0
Entradas primárias / black-box casadas	8
Unmatched (reference / implementation)	0 / 0

```

***** Matching Results *****
81 Compare points matched by name
0 Compare points matched by signature analysis
0 Compare points matched by topology
8 Matched primary inputs, black-box outputs
0(0) Unmatched reference(implementation) compare points
0(0) Unmatched reference(implementation) primary inputs, black-box outputs
*****

```

Figura 11 - Resultado do match: 81 compare points casados, 0 unmatched

O guidance do SVF foi integralmente aproveitado: das 18 operações registradas pelo Design Compiler, todas as 18 foram aceitas e nenhuma rejeitada. As 8 operações reg_constant correspondem à propagação de constantes prevista na literatura de equivalência (bits que nunca mudam de valor), resolvidas automaticamente pelo guidance.

O relatório report_svf_operation -status rejected resultou vazio: nenhuma operação do SVF foi rejeitada e, portanto, nenhuma exigiu investigação manual. Este é o resultado esperado quando a biblioteca, o RTL e a netlist são consistentes com o SVF gerado na mesma rodada de síntese.

Operação SVF	Aceitas	Rejeitadas
change_names	5	0
environment	4	0
reg_constant	8	0
ungroup	1	0
Total	18	0

```

***** Guidance Summary *****

```

		Status			
Command		Accepted	Rejected	Pending Unprocessed	Total
change_names	:	5	0	0	5
environment	:	4	0	0	4
reg_constant	:	8	0	0	8
ungroup	:	1	0	0	1
Total	:	18	0	0	18

Figura 12 - Guidance Summary: 18 operações aceitas, 0 rejeitadas (rodada com -no_autoungroup)

8.4 Verify e sign-off da netlist principal

O comando verify provou a equivalência lógica de cada par de pontos casados. O veredito final para a netlist principal (com -no_autoungroup) foi SUCCEEDED, com 81 pontos passando e nenhum failing:

```
***** Verification Results *****
Verification SUCCEEDED
  ATTENTION: synopsys_auto_setup mode was enabled.
              See Synopsys Auto Setup Summary for details.
  ATTENTION: Inconsistency detected in design data with SVF.
              See Design Consistency Check Summary for details.
-----
Reference design: r:/WORK/vending_top
Implementation design: i:/WORK/vending_top
81 Passing compare points
-----
Matched Compare Points   BBPin   Loop   BBNet   Cut   Port   DFF   LAT   TOTAL
-----
Passing (equivalent)      0       0     0       0    21     60     0     81
Failing (not equivalent)  0       0     0       0     0      0      0      0
*****
```

Figura 13 - Veredito de equivalência da netlist principal: Verification SUCCEEDED

Com o comando verify, confirmamos que a lógica bate perfeitamente. O resultado final para a netlist principal (usando -no_autoungroup) foi um sucesso: 81 pontos verificados sem nenhuma falha.

O modo "Synopsys Auto Setup" rodou normalmente durante a verificação. Mesmo com os avisos FM-622 sobre a falta do "guide_hier_map", o guidance funcionou sem problemas, já que mantivemos a hierarquia preservada pelo comando -no_autoungroup, garantindo a validade da verificação.

Sobre a versão da ferramenta: o roteiro sugere usar "report_status -verbose", mas na versão que estamos usando (X-2025.06-SP3), essa opção não existe. Usamos apenas "report_status", que já entrega todos os dados necessários (incluindo o resumo do setup e a tabela de comparação), o que resolveu sem complicações.

Por fim, rodamos os relatórios de pontos não casados e falhas ("report_unmatched_points" e "report_failing_points") e ambos vieram vazios. Ou seja, não tivemos problemas de casamento ou divergências lógicas; o arquivo SVF conseguiu mapear tudo corretamente, dispensando ajustes manuais ou análises adicionais de cones lógicos.

8.5 Comparação: efeito de -no_autoungroup

A verificação foi repetida para uma segunda síntese, desta vez sem a opção -no_autoungroup (permitindo ungroup automático dos módulos). Cada netlist utilizou o seu próprio SVF, gerado na respectiva rodada de compile_ultra, pois cada rodada realiza otimizações diferentes. Ambas as rodadas chegaram a **SUCCEEDED** com 81 pontos equivalentes; a diferença está na quantidade de transformações registradas no SVF:

Métrica	Rodada A (-no_autoungroup)	Rodada B (ungroup livre)
Compare points	81	81
Unmatched points	0	0
Operações change_names	5	5
Operações environment	4	5
Operações reg_constant	8	8
Operações ungroup	1	5
Total de operações SVF (accepted)	18	23
Operações rejeitadas	0	0
Tempo de CPU (verify)	4,31 s	3,47 s
Tempo total (elapsed)	33 s	3 s
Veredito final	SUCCEEDED	SUCCEEDED

Dá para notar que manter a hierarquia (-no_autoungroup) deixou o SVF bem mais enxuto, com apenas 18 operações. Já com o ungroup livre, o compilador saiu fundindo módulos e gerou 23 operações. O ponto principal aqui é que, mesmo com essa bagunça extra na lógica, o SVF deu conta de tudo sozinho: nenhuma operação foi rejeitada e não precisamos casar nenhum ponto na mão. Isso prova que o fluxo com SVF é muito mais garantido do que tentar um match puramente estrutural.

Sobre a performance, o tempo de CPU ficou na mesma faixa (cerca de 4 segundos), o que faz sentido já que o hardware é o mesmo. A variação no tempo total (elapsed) da primeira rodada foi só oscilação do servidor na hora do teste. No dia a dia, o que importa é que o Formality resolveu os dois casos rápido e sem erro.

```

***** Guidance Summary *****
                        Status
Command      Accepted  Rejected  Pending  Unprocessed  Total
-----
change_names :         5         0         0         0         5
environment  :         5         0         0         0         5
reg_constant :         8         0         0         0         8
ungroup      :         5         0         0         0         5
-----
Total        :        23         0         0         0        23

```

Figura 14 - Guidance Summary da rodada sem -no_autoungroup: 23 operações aceitas, 0 rejeitadas

```

***** Verification Results *****
Verification SUCCEEDED
  ATTENTION: synopsys_auto_setup mode was enabled.
              See Synopsys Auto Setup Summary for details.
  ATTENTION: Inconsistency detected in design data with SVF.
              See Design Consistency Check Summary for details.

-----
Reference design: r:/WORK/vending_top
Implementation design: i:/WORK/vending_top
81 Passing compare points
-----
Matched Compare Points   BBPin   Loop   BBNet   Cut   Port   DFF   LAT   TOTAL
-----
Passing (equivalent)      0       0       0       0    21    60     0    81
Failing (not equivalent)  0       0       0       0     0     0     0     0
*****

```

Figura 15- Veredito de equivalência da netlist ungrouped: Verification SUCCEEDED

8.6 Discussão

A principal conclusão aqui é que o uso do arquivo SVF é o que torna a verificação formal um processo realmente confiável e automático. Na prática, sem o SVF, você fica refém da análise puramente estrutural da ferramenta, o que vira uma dor de cabeça enorme quando o sintetizador renomeia sinais ou faz otimizações como o ungroup.

Ao carregar o SVF e manter o modo auto_setup ativo, nós basicamente entregamos o mapa do que o Design Compiler fez para o Formality, evitando qualquer necessidade de intervenção manual ou de tentar casar pontos na mão. Para um design desse tamanho, o resultado foi um sign-off limpo, sem erros ou pontos não casados, confirmando que a netlist sintetizada é logicamente idêntica ao RTL original. Definitivamente, esse é o fluxo mais seguro para garantir que a lógica permaneceu íntegra após a síntese.